

Universitatea din Pitești
Facultatea de Electronică și Calculatoare

Unified Modelling Language(UML)

Lect.univ.dr. Adrian Țurcanu
Lect.univ.dr. Cristina Tudose

Cuprins

- 1 Introducere
- 2 Diagrama cazurilor de utilizare
- 3 Diagrame de clase și diagrame de obiecte
- 4 Diagrame de interacțiune
- 5 Diagrame de stare

Ce este un model?

- **Modelarea** este o etapă esențială în orice proiect software.
- Un **model** reprezintă o abstractizare a unui sistem, într-un anumit context, pentru un scop bine precizat.
- Modele sunt elaborate în etape premergătoare dezvoltării software și conțin doar aspecte esențiale ale sistemelor.
- Modele sunt utilizate, în special, înainte de codificare pentru a verifica dacă: sunt acoperite cerințele clienților, funcțiile prevăzute sunt completa, arhitectura este robustă și extensibilă, dar și după codificare pentru verificarea și validarea sistemului.

UML

- UML este un limbaj de modelare orientat pe obiecte apărut din necesitatea unei standardizări a elementelor de modelare.
- Prima versiune UML a fost publicată în 1996 (Booch, Rumbaugh, Jacobson) și a fost urmată de alte versiuni:
 - 1997: UML 1.1
 - 1998: UML 1.2
 - 1999: UML 1.3
 - 2002: UML 1.4
 - 2004: UML 2.0
 - 2007: UML 2.1.2

Caracteristici ale limbajului UML

- permite modelarea sistemelor software complexe prin intermediul mai multor categorii de diagrame
- modelarea este realizată pe diferite niveluri de detaliere și în diverse etape ale procesului
- folosește, în general, elemente independente de limbajele de implementare, dar sunt admise și construcții preluate din diferite limbaje concrete de implementare
- este bazat pe reprezentarea conceptelor de bază ale **modelului obiect**: abstractizarea, modularitatea, încapsularea, moștenirea
- are un grad de generalitate mai ridicat decât modelul obiect putând fi folosit și pentru modelarea sistemelor de altă natură

Concepte ale modelului obiect

- **Abstractizarea** - tehnică de reprezentare simplificată a unui sistem având la bază proprietățile cele mai relevante ale acestuia, din punct de vedere al scopului propus.
- În POO presupune definirea claselor și a obiectelor (instanțe ale claselor).
- **Modularitatea** - un sistem este alcătuit din mai multe module, ceea ce facilitează dezvoltarea, modificarea și întreținerea acestuia.
- În POO modulele sunt alcătuite din clase între care există diferite legături (moștenire, asociere, dependență).

Concepte ale modelului obiect

- **Încapsularea** - ascunderea implementării.
- În POO elementele unei clase sunt separate în două părți:
 - structura - atribute și implementarea metodelor
 - interfața - prototipurile metodelor publice prin care poate fi accesat un obiect.
- Obiectele sunt accesate doar prin intermediul metodelor publice, iar modificările nu afectează interfața.
- **Moștenirea** - presupune crearea unor ierarhii de clase printr-un proces de specializare sau generalizare:
 - clasa de bază (superclasa) - clasa mai generală
 - clase derivate (subclase) clase particulare, specializate care moștenesc atribute și metode ale superclasei

Elemente de modelare

Principalele elementele de modelare definite în UML pot fi împărțite în 3 categorii:

- Modelare comportamentală: diagrame ale cazurilor de utilizare, diagrame de interacțiune, diagrame de stări, diagrame de activități
- Modelare structurală: diagrame de clase, diagrame de obiecte, inferfețe, pachete
- Modelare arhitecturală: diagrame de componente, diagrame de distribuție

Scenariu - exemplu

Se consideră un sistem de gestiune electronică a unui lanț de farmacii. Există trei categorii de utilizatori ai sistemului: farmaciști, administratori și clienți. Clienții pot fi clienți obișnuiți sau clienți care dețin un card de reduceri. Pentru clienții posesori de card, sistemul reține o serie de informații personale. Farmaciștii pot adauga clienți în sistem, gestionează stocul de medicamente și vânzarea acestora. Clienții pot cumpăra medicamente, pot comanda medicamente care nu sunt pe stoc, pot cere să intre în posesia unui card sau își pot anula cardul. Administratorii controlează sistemul în ansamblul său având posibilitatea de a adăuga sau elimina farmacii sau farmaciști din cadrul acestora, de a modifica lista medicamentelor disponibile.

Modelul cazurilor de utilizare

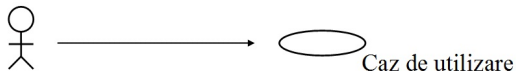
- Introdus de Jacobson în 1992, modelul cazurilor de utilizare descrie comportamentul sistemului din punct de vedere al utilizatorului.
- Reprezintă un instrument folosit în mod frecvent în specificarea sistemelor informatice în fazele inițiale ale procesului de definire a cerințelor utilizatorilor.
- Reprezintă:
 - sistemul (incluzând componente și descrierea acestora)
 - utilizatorii (elemente externe sistemului)
- Cuprinde:
 - Diagrama cazurilor de utilizare
 - Descrierea cazurilor de utilizare

- Modelul cazurilor de utilizare include:
 - actori
 - cazurile de utilizare
 - scenarii
 - diagramele de cazuri de utilizare.

Actori

- Un actor este un rol pe care o entitate externa îl joacă în raport cu un sistem.
- Actorii se determină observând utilizatorii direcți ai sistemului, cei care sunt responsabili de exploatarea sau de interogarea sa.
- Același individ din viața reală poate juca rolul mai multor actori ai modelului.
- Mai mulți indivizi pot juca același rol, caz în care li se asociază același actor.
- Un actor poate fi de asemenea un echipament extern sistemului sau un alt sistem.
- Actorii se reprezintă în diagrame sub forma unor “omuleți” care declanșează cazuri de utilizare.

Exemplu de actor



- Linia care conectează un actor de un caz de utilizare se numește **legătură de comunicare**.
- Actori în exemplul cu lanțul de farmacii: Administrator, Farmacist, Client, ClientCuCard.

Categorii de actori

- **Actorii primari** sunt beneficiarii direcți ai sistemului și sunt cei care inițiază cazurile de utilizare.
- **Actorii secundari** sunt cei cu ajutorul cărora se realizează cazul de utilizare. Actorii secundari nu inițiază cazul de utilizare, dar participă la realizarea acestuia.
- De obicei, doar actorii primari sunt reprezentați în diagrama cazurilor de utilizare, dar toți actorii sunt menționați în descrierea cazului de utilizare.

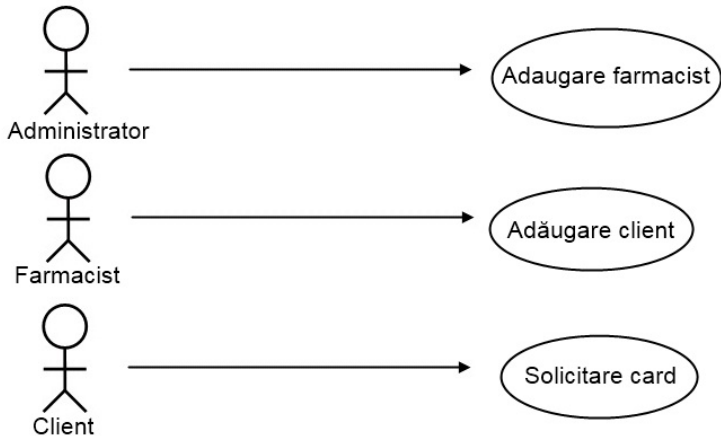
Identificarea actorilor

- Identificarea actorilor presupune identificarea tuturor rolurilor pe care le au utilizatorii sistemului.
- Permite precizarea limitelor sistemului într-o manieră progresivă: vagi la început, ele se precizează pe măsura elaborării diferitelor cazuri de utilizare.
- Este o activitate deosebit de importantă pentru că permite stabilirea a ceea ce trebuie să realizeze viitorul sistem, ceea ce face parte din sistemul dezvoltat și ceea ce nu face parte din el.

Cazuri de utilizare

- Un caz de utilizare este o unitate coerentă de funcționalitate ce descrie dialogul dintre un utilizator și sistem.
- Un caz de utilizare înglobează un set de cerințe ale sistemului care reies din specificațiile inițiale, urmând ca acestea să fie rafinate pe parcurs.
- Cazurile de utilizare pot avea complexități diferite, în funcție de setul de cerințe cărora corespund.
- Cazurile de utilizare descriu interacțiuni potențiale ale utilizatorilor cu sistemul, fără a intra în detalii ale fiecărui scenariu.

Exemple de cazuri de utilizare



Identificarea cazurilor de utilizare - 2 metode

- o metodă bazată pe actori:
 - se identifică actorii sistemului
 - pentru fiecare actor se identifică cazurile de utilizare pe care le inițiază sau la care participă
- o metodă bazată pe evenimente:
 - se identifică evenimentele externe
 - se face legătura dintre aceste evenimente și actori

Șablon pentru descrierea cazurilor de utilizare - Derek Coleman (1998)

Caz de utilizare	<i>Identificatorul și numărul de referință al cazului de utilizare, istoria modificărilor</i>
Descriere	<i>Scopul în care e folosit cazul de utilizare și sursa cerințelor funcționale</i>
Actori	<i>Lista actorilor implicați</i>
Ipoteze	<i>Condiții ce trebuie să fie adevărate ca un caz de utilizare să se termine cu succes</i>
Pași	<i>Interacțiunile dintre actori și sistem care sunt necesare pentru a atinge scopul în care e folosit cazul de utilizare</i>
Alternative (opțional)	<i>Orice alternativă întâlnită în pașii cazului de utilizare</i>
Cerințe non-funcționale (opțional)	<i>Lista cerințelor non-funcționale pe care trebuie să le îndeplinească un caz de utilizare</i>
Probleme	<i>Lista problemelor care rămân să</i>

Scenarii

- Un scenariu este o secvență de pași care descrie o posibilă interacțiune dintre un sistem și un actor.
- Exemplu - scenarii de adăugare client:

[1] Client inexistent - Date OK:

- Farmacistul deschide un formular de adăugare a unui client nou și completează date solicitate.
- Sistemul validează datele introduse și verifică existența clientului.
- Datele sunt valide și clientul nu exista anterior și se finalizează procedura de înregistrare

Scenarii

[2] Client inexistent - Date eronate:

- Farmacistul deschide un formular de adăugare a unui client nou și completează date solicitate.
- Sistemul validează datele introduse și verifică existența clientului.
- Datele sunt invalide(de exemplu, CNP cu are 13 cifre) și se cere corectarea datelor.
- Se corectează datele eronate și se finalizează procedura de înregistrare.

Scenarii

[3] Client deja existent:

- Farmacistul deschide un formular de adăugare a unui client nou și completează date solicitate.
- Sistemul validează datele introduse și verifică existența clientului.
- Clientul este deja în sistem, se trimite un mesaj corespunzător și farmacistul este întrebat dacă înlocuiește sau actualizează clientul.
- Farmacistul finalizează modificările.

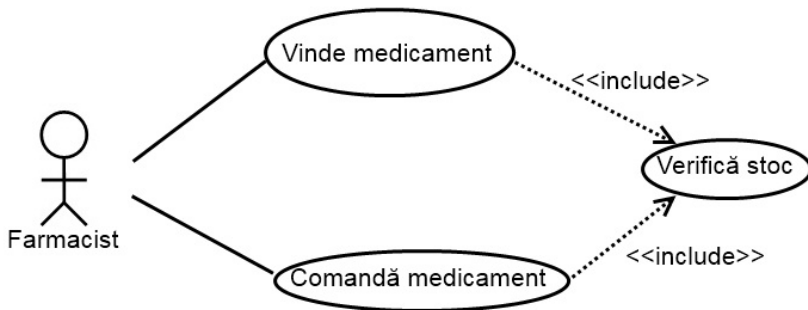
Relații între cazurile de utilizare

- Diagramele cazurilor de utilizare sunt folosite pentru a reprezenta relațiile dintre cazurile de utilizare ce descriu un sistem.
- Există trei tipuri de relații între cazurile de utilizare:
 - relația de incluziune
 - relația de extindere
 - relația de generalizare

Relația de incluziune

- Două cazuri de utilizare pot avea o componentă comună, care poate fi utilizată la definirea fiecăruia dintre ele.
- Componenta comună este tot un caz de utilizare care este legat prin relația *<< include >>* de fiecare din cazurile care îl utilizează.
- Relația de incluziune se folosește pentru a evita desrierea multiplă a aceluiași flux de evenimente.
- Scenariile care reprezintă instanțe ale cazului de bază conțin subsenarii care sunt instanțe ale cazului de utilizare inclus.
- Avantaje: evitarea redundanțelor, simplificare descrierii unor cazuri de utilizare, reutilizarea unor componente ale sistemului.

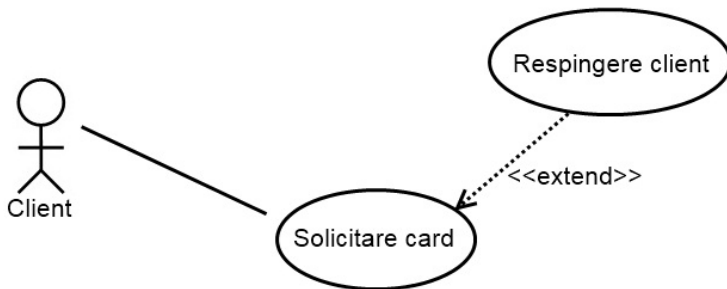
Exemplu - incluziune



Relația de extindere

- Un caz de utilizare poate extinde, în anumite condiții, comportarea reprezentată printr-un alt caz.
- Relația de extindere se folosește, în general, pentru reprezentarea unor situații excepționale.
- Comportamentul cel mai frecvent întâlnit este cel descris de cazul general, iar situațiile rar întâlnite sunt considerate extensii ale acestuia.

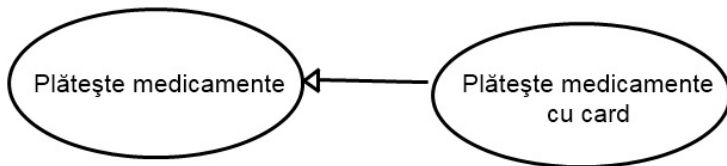
Exemplu - extindere



Relația de generalizare

- Are o semnificație similară cu cea a relației de generalizare între clase.
- Poate fi folosită atât între actori cât și între cazuri de utilizare.
- Generalizarea între actori arată că un actor moștenește structura și comportamentul unuia sau mai mulți actori.
- Generalizarea între cazuri de utilizare arată că un caz de utilizare (“copil”) moștenește comportamentul unuia sau mai multor cazuri de utilizare (“părinte”).
- Cazul de utilizare copil poate adăuga noi aspecte ale comportării sau poate redefini parțial comportarea părintelui.
- Relația de generalizare se poate confunda în anumite situații cu cea de extindere.

Exemplu - generalizare



Diagrame de clase și diagrame de obiecte

- Diagramele de clase și diagramele de obiecte se folosesc pentru modelarea structurală, care captează aspectele statice ale sistemului: elementele componente și legăturile dintre acestea
- **Diagrama claselor** definește modelul structural al unui sistem prin intermediul unui graf, ale cărui noduri reprezintă clasele, iar arcele reprezintă legăturile dintre clase.
- **Diagrama obiectelor** reprezintă tot prin intermediul unui graf structura statică a sistemului la un moment dat, folosind instanțe ale claselor (obiecte) și instanțe ale legăturilor dintre clase existente în acel moment.
- Diagrama obiectelor este mai puțin generală decât diagrama claselor și de aceea este mai puțin utilizată.

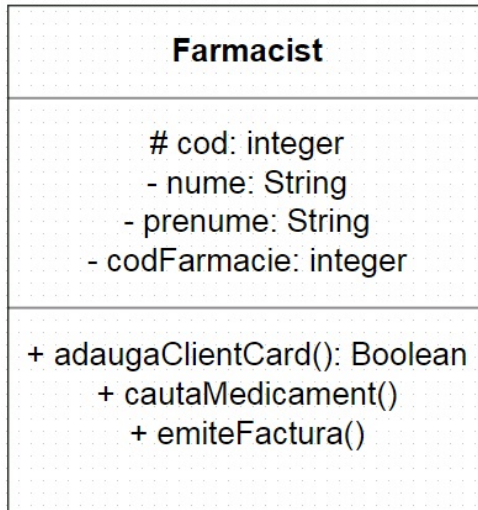
Diagrame de clase

- O clasă reprezintă o mulțime de obiecte care au: proprietăți comune(atribute), un comportament comun(operații), relații comune cu alte obiecte și o semnificație comună.
- În UML, o clasă este reprezentată printr-un dreptunghi alcătuit din trei compartimente care conțin: numele clasei, attributele și operațiile.
- În etapele inițiale, clasele conțin doar numele, attributele și eventual unele operații.
- Reprezentarea detaliată este realizată ulterior și conține pentru fiecare clasă detalii ca vizibilitatea informațiilor sau lista de parametri a fiecărei operații.

Vizibilitatea atributelor și a metodelor

- **private**(-): accesibilitate doar în interiorul clasei
- **public**(+): accesibilitate la nivelul întregului sistem
- **protected**(#): accesibilitate în arborele de moștenire
- **package**(~): accesibilitate în interiorul pachetului care conține clasa

Exemplu de clasă



Relații între clase

Între clase există mai multe tipuri de relații:

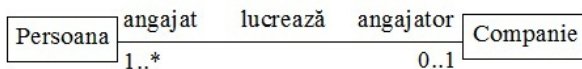
- asociere
- generalizare
- dependență
- realizare

Asocierea

- **Asocierea** reprezintă abstractizarea unei legături structurale între clase.
- Asocierea claselor este o consecință a interacțiunii dintre obiecte ale acestora.
- Asocierile sunt reprezentate prin verbe.
- Situații în care două clase sunt asociate:
 - un obiect din clasa A trimite un mesaj către un obiect din clasa B ;
 - un obiect din clasa A creează un obiect din clasa B ;
 - un obiect din clasa A are un atribut ale cărui valori sunt obiecte din clasa B .

Roluri

- Înțelegerea rolurilor pe care obiectele le au în cadrul unei asocieri are rolul de clarifica necesitatea asocierii.
- Un rol exprimă ce reprezintă obiectele unei clase în raport cu obiectele celeilalte.
- Rolurile se folosesc, de obicei, atunci când între două clase există mai multe asocieri.

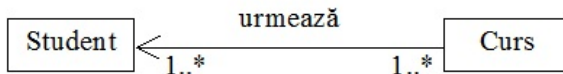


Multiplicitatea unei asocieri

- Asocierile sunt caracterizate de **multiplicități** care arată câte obiecte ale unei clase pot fi legate, la un moment dat de un obiect al celeilalte clase.
- Multiplicitățile pot fi:
 - un număr fixat (de exemplu 1);
 - un număr oarecare, caz în care folosim simbolul “*” (i se atribuie sensul “mai multe”);
 - o mulțime de numere întregi: 0..1, 2..7, 1..*.
- Multiplicitatea unei asocieri exprimă o constrângere valabilă pe tot parcursul existenței claselor implicate.

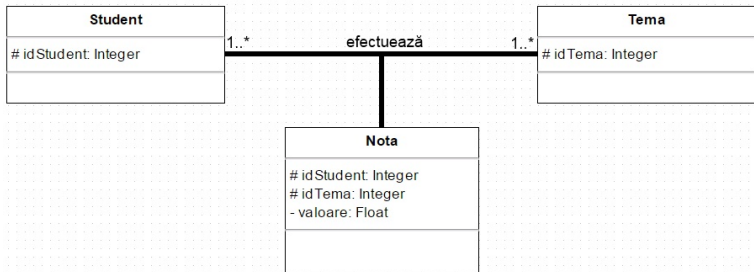
Navigabilitatea

- În general, asocierile între clase sunt considerate bidirecționale.
- Plasarea unei săgeți la unul din capetele liniei ce reprezintă asocierea arată că este posibil să se transmită mesaje în direcția indicată.
- Orientarea asocierii nu înseamnă că nu este posibilă și navigația în sens invers, ci că aceasta nu este importantă sau este greu de realizat.
- Navigabilitatea este specificată explicit doar dacă este foarte importantă pentru proiect.



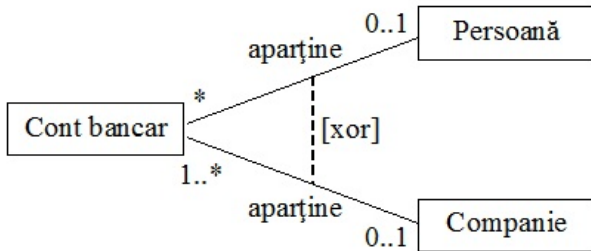
Asocieri atributate

- Asocierile pot fi caracterizate prin **atribute**, caz în care se numesc **asocieri atributate**.
- Asocierile atributate se reprezintă prin clase cu atribute și operații proprii atașate asocierii printr-o linie punctată.



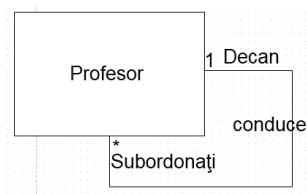
Asocieri mutual exclusive

- Două asocieri a căror existență se exclude reciproc se numesc **asocieri mutual exclusive**.



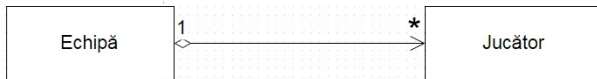
Asocieri reflexive

- O asociere poate lega o clasă de ea însăși, caz în care se numește **asociere reflexivă**.
- În acest caz, rolurile sunt esențiale pentru claritatea diagramei.



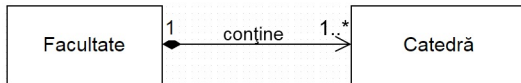
Agregarea

- **Agregarea** este un tip particular de asociere folosit pentru a exprima o relație de tip parte-întreg.
- Agregarea se reprezintă prin intermediul unui romb gol plasat alături de clasa agregat.
- Fiecare din părțile clasei agregat au o existență independentă.



Compunerea

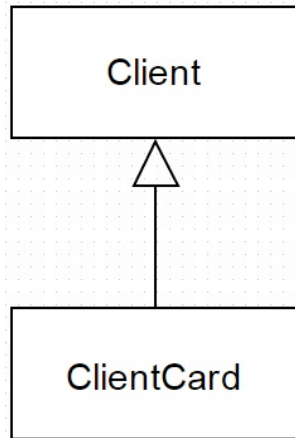
- **Compunerea** este un tip particular de agregare ce implică o conținere fizică a părților în întreg și o simultaneitate a duratelor de viață ale părților și ale întregului.
- Crearea sau distrugerea obiectului agregat implică crearea, respectiv distrugerea părților componente.
- Compunerea se reprezintă prin intermediul unui romb plin plasat alături de clasa agregat.
- În cazul compunerii, o parte poate fi conținută de un singur întreg, deci multiplicitatea asocierii la extremitatea clasei agregat este 1.



Generalizarea

- Generalizarea este o relație între o clasă părinte (super-clasă) și o clasă copil(sub-clasă).
- De obicei, elementele comune ale unui ansamblu de clase se grupează într-o clasă mai generală(super-clasa).
- Clasele sunt ordonate apoi într-o ierarhie în care relația de generalizare este reprezentată printr-o săgeată orietată spre super-clasă.
- Sub-clasele moștenesc structura și comporamentul super-clasei; în plus, se pot adăuga attribute, operații și relații sau se poate schimba comportamentul din super-clasă (suprascrisiere).
- Un obiect al unei super-clase, poate fi substituit cu un obiect al unei sub-clase, dar nu și reciproc.

Exemplu generalizare



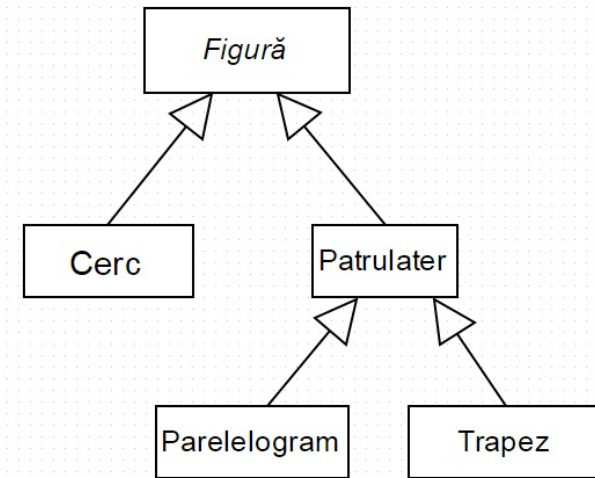
Generalizarea

- O clasă copil poate moșteni comportamentul unui singur părinte (**moștenire simplă**) sau al mai multor părinți (**moștenire multiplă**).
- Moștenire multiplă este, în general evitată pentru că poate crea confuzii. Aceasta poate fi înlocuită cu moștenirea simplă în care părintele este obținut prin agregarea mai multor clase părinte.
- O ierarhie de clase se reprezintă prin intermediul unui **arbore de moștenire** sau a unui **graf de moștenire**.

Clase abstracte

- Uneori, anumite clase sunt create doar ca surse de moștenire pentru alte clase. Astfel de clase se numesc **clase abstracte** și nu pot fi instanțiate.
- Clasele abstracte nu furnizează, în general, implementarea operațiilor.
- Operațiile sunt implementate în sub-clase, numite clase concrete.
- Numele claselor abstracte se scriu cu italic în diagramele UML.
- În arborele de moștenire, clasele abstracte sunt noduri rădăcină, iar cele concrete sunt noduri frunză.

Exemplu clasă abstractă



Dependențe și constrângeri

- O **dependență** este o relație între două clase în care modificarea uneia poate implica modificarea celeilalte.
- Cel mai frecvent caz de dependență este relația dintre o clasă și o altă clasă pe care aceasta o folosește ca parametru într-o operație.
- O **constrângere** este o condiție pe care orice implementare a diagramei claselor trebuie să o satisfacă.
- Constrângerile se scriu între acolade.

Interfețe

- În UML, o **interfață** este o colecție de operații pe care trebuie să le furnizeze o clasă sau o componentă.
- Interfețele nu au atribute și nu conțin implementări ale operațiilor.
- O interfață este reprezentată într-o diagramă de clase tot printr-un dreptunghi, din care lipsește zona dedicată atributelor și care are scris în partea de sus cuvântul cheie "interface".
- O interfață poate reuni operații din mai multe clase și operațiile dintr-o interfață pot fi implementate de mai multe clase.
- Interfețele sunt folosite, în general, în limbaje de programare care nu permit moșteniri multiple.
- În astfel de situații, deși poate moșteni doar o super-clasă, o

Realizarea modelului claselor

- Clasele provin, în general, din două surse:
 - obiecte concrete: medicament, farmacie, etc.
 - roluri: farmacist, administrator, client, etc.
- Principii de realizare:
 - clasele trebuie să fie bine delimitate, iar gradul de dependență dintre acestea să fie cât mai redus;
 - clasele trebuie alese astfel încât să fie ușor de adaptat la modificarea cerințelor.
- Un model ideal conține clase care reprezintă obiecte stabile în timp, independente pe cât posibil de implementarea curentă.

Realizarea modelului claselor

- Strategii de indentificare a claselor:
 - conduse de date: indentificarea datelor din sistem și împărțirea lor în clase(ex. tehnica indentificării substantivelor)
 - conduse de responsabilități: indentificarea responsabilităților și împărțirea lor în clase(ex. tehnica cardurilor CRC)
- Tehnica indentificării substantivelor presupune selectarea substantivelor care apar în cerințele sistemului și eliminarea celor inadecvate pentru a reprezenta clase.
- Un substantiv poate fi inadecvat dacă reprezintă: un obiect singular, un sinonim al altui substantiv, o operație, un atribut, etc.

Carduri CRC

- Cardurile CRC(Class, Responsibilities, Collaboration) sunt folosite pentru rafinarea designului.
- Un card CRC conține: numele clasei, responsabilitățile clasei, colaboratorii clasei.
- Responsabilitățile clasei descriu scopul pentru care există clasa respectivă.
- Responsabilitățile pot fi percepute ca un rezumat al operațiilor pe care clasa le poate realiza.

Exemplu card CRC

<i>Farmacist</i>	
Responsabilități	Colaboratori
Caută, vinde sau comandă medicamente	Medicament
Adaugă date clienți cu card	ClientCard

Tool online free pentru diagrame UML: <https://www.draw.io>.

Diagrame de interacțiune

- Sunt folosite pentru a evidenția modul în care mai multe obiecte interacționează pentru a executa un task.
- Ilustrează modul în care sistemul realizează un caz de utilizare sau un scenariu al unui caz de utilizare.
- Există două tipuri de diagrame de interacțiune:
 - diagrame de colaborare
 - diagrame de secvență.

Diagrame de colaborare

- Entitățile care interacționează pentru realizarea unui task împreună cu legăturile dintre acestea formează o **colaborare**.
- O colaborare cuprinde:
 - **Obiecte**: se reprezintă prin dreptunghiuri care conțin `numeObiect:NumeClasă`;
 - **Legături**: sunt instanțe ale legăturilor existente între clasele din care fac parte obiectele respective;
 - **Actori**: se reprezintă ca și în cazurile de utilizare. Pot exista mai mulți actori, dar unul singur inițiază cazul de utilizare.
 - **Mesaje**: se reprezintă pe liniile ce leagă obiectele, direcția de transmitere fiind indicată printr-o săgeată.

Interacțiuni procedurale

- După transmiterea unui mesaj, obiectul emitent așteaptă un răspuns înainte să-și continue funcționarea.
- Un obiect este **activat** atunci când primește un mesaj.
- Acesta trebuie să trimită apoi un răspuns obiectului care a trimis mesajul, iar între timp el poate să lucreze sau să trimită mesaje altor obiecte pentru a le face pe acelea să lucreze.
- Dacă obiectul trimite un mesaj, el va fi în continuare activ, dar nu va putea să lucreze până când nu va primi răspunsul la acest mesaj.
- Odată cu transmiterea unui mesaj se transmite și controlul către obiectul receptor.

- Se creează astfel o stivă de activări, fiecare activare fiind asociată unui obiect care a primit un mesaj la care nu a răspuns încă.
- Mesajele sunt numerotate după următoarele reguli:
 - primul mesaj transmis de la un obiect la altul este nume­to­rat cu 1, următorul cu 2, etc.
 - numărul mesajului primit de un obiect este folosit ca prefix pentru toate mesajele trimise de acesta până când răspunde la mesajul inițial.

Diagrame de secvență

- Sunt diagrame de interacțiune care pun în evidență transmiterea mesajelor de-a lungul timpului.
- Obiectele și actorii sunt repartizați la capătul de sus al unor linii punctate, care reprezintă **linia de viață** a obiectelor.
- Scurgerea timpului este reprezentată în cadrul diagramei de sus în jos.
- Fiecare mesaj se reprezintă printr-o săgeată de la linia de viață a obiectului emițător, la linia de viață a destinatarului.
- Timpul în care un obiect este activat este reprezentat printr-un dreptunghi care acoperă linia sa de viață pe o anumită porțiune.
- Opțional, se pot reprezenta prin linii punctate și răspunsurile la mesaje.

Crearea și distrugerea obiectelor

- Crearea și distrugerea unui obiect pot fi reprezentate într-o diagramă de colaborare prin intermediul cuvintelor cheie `<<create>>` și, respectiv `<<destroy>>`.
- În diagramele de secvență crearea unui obiect este evidențiată prin plasarea acestuia în josul paginii, poziția sa corespunzând cu momentul în care a fost creat obiectul. Distrugerea unui obiect este marcată printr-un "X".

Iterații

- Trimiterea repetată a unui mesaj este marcată prin intermediul unui asterisc poziționat după numărul mesajului.
- Acest asterisc este urmat uneori de o **clauză de iterație** care determină de câte ori va fi transmis mesajul.
- Clauze de iterație poate avea mai multe forme:
 - $[n < const]$: mesajul este transmis atâta timp cât n este mai mic decât constanta specificată;
 - $not\ x$: mesajul este transmis până când condiția x devine adevărată;
 - $[i := 1..10]$: mesajul este transmis de 10 ori;
 - $a\ \text{în}\ A$: mesajul este transmis pentru fiecare a din A .

Concurența

Există mai multe situații în care sistemele sunt concurente(lucrează mai mult de un obiect):

- un obiect transmite simultan mesaje către mai multe obiecte, acestea fiind activate simultan
- un actor sau un obiect activ trimite un mesaj la un anumit moment deși nu el este cel care lucrează în acel moment ci un alt obiect din sistem;
- un obiect trimite un mesaj care face ca alt obiect să lucreze fără a-și înceta activitatea(mesaje asincrone).

Tipuri de mesaje

- **mesaje sincrone**: obiectul care trimite un mesaj pierde controlul până când primește un răspuns.
- **mesaje de răspuns**: răspunsuri la mesaje sincrone
- **mesaje asincrone**: obiectul care trimite un mesaj rămâne activ și poate trimite și alte mesaje
- **mesaje simple**: obiectul care trimite mesajul pierde controlul, iar următorul mesaj este transmis de obiectul destinatar.

Diagrame de stare

- Obiectele unei clase pot reacționa diferit la primirea unui mesaj, în funcție de starea acestora, adică de valorile curente ale atributelor.
- **Diagramele de stare** (state machine diagrams) descriu schimbările de stare ale unui obiect ca urmare a unui mesaj sau a unui alt eveniment pe care îl recepționează.
- Diagramele de stare au la bază analiza diagramelor de interacțiune pentru a identifica obiectele care, pe parcursul vieții lor, reacționează în mod diferit la mesajele pe care le primesc.
- O diagramă de stare poate fi interpretată ca un automat cu stări finite(FSM).

Elemente componente

- **semnul de început**: este un disc negru din care pornește o săgeată către **starea inițială** a obiectului, aceasta fiind starea implicită a obiectului atunci când acesta este creat
- **stări** reprezentate prin dreptunghiuri cu colțuri rotunjite
- **tranziții între stări**, reprezentate prin săgeți
- **evenimente** care declanșează tranzițiile între stări
- **semnul de sfârșit**: este un disc negru înconjurat de un cerc în care intră o săgeată care pornește dintr-o stare finală a obiectului. O astfel de stare apare dacă obiectul este distrus la un moment dat.

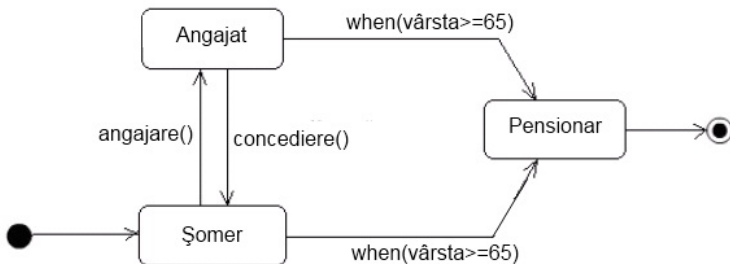
Exemplu



Evenimentele pot fi:

- un semnal recepționat: o excepție, o notificare, un eveniment generat de interacțiunea cu utilizatorul
- operații ale clasei obiectului
- o condiție predefinită care este îndeplinită la un moment dat
- trecerea unei perioade de timp

Exemplu

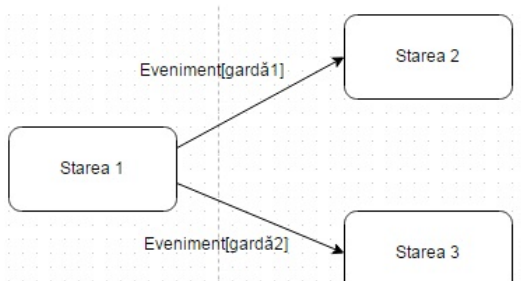


Stările diagramei

- O stare a unui obiect este o mulțime de configurații ale obiectului care se comportă similar la apariția unui eveniment.
- O stare poate fi identificată prin mulțimea valorilor atributelor obiectului la un moment dat.
- O diagramă de stare trebuie să fie minimală, adică nu trebuie să conțină stări echivalente din punct de vedere comportamental.
- De exemplu, în diagrama de stare a unui televizor nu asociem câte o stare fiecărui canal pe care televizorul ar fi aprins ci o singură stare în care televizorul se consideră aprins pe un anumit canal.

Gărzi

- Declanșarea tranzițiilor între stări poate fi validată și prin intermediul unor condiții booleene numite **gărzi**, atașate evenimentelor. În acest caz vorbim despre **tranziții condiționate**.
- Dacă mai multe tranziții pot fi declanșate de același eveniment, gărzile trebuie să fie mutual exclusive.



Declanșarea programată

- O tranziție între stări poate fi declanșată după o anumită perioadă de timp.
- Această perioadă se specifică pe săgeata corespunzătoare tranziției și este marcată prin cuvântul cheie **after**.

